

Lab 3. π approximation

In this "mini-project", you will use the Monte Carlo method to approximate the value of π , evaluate the approximation errors, and analyze how the accuracy changes with different numbers of random samples.

```
In [5]: import numpy as np
```

Part 1 - π approximation

Consider a square of side length 2 centered at the origin

$$[-1, 1] \times [-1, 1]$$

and an inscribed circle of radius $R = 1$, also centered at the origin

$$\{(x, y) \in \mathbb{R}^2 : x^2 + y^2 \leq 1\}.$$

The area of the square S is

$$A(\text{square}) = 4.$$

The area of the circle C is:

$$A(\text{circle}) = \pi.$$

The probability that such a point, chosen uniformly within the square, lies inside the circle is exactly equal to

$$P(C) = \frac{A(\text{circle})}{A(\text{square})} = \frac{\pi}{4}.$$

This gives us a simple method to approximate π .

π approximation Algorithm:

1. Fix n .
2. Uniformly sample n points (x_i, y_i) in the square $[-1, 1] \times [-1, 1]$.
3. Count the number K of points that satisfy $x_i^2 + y_i^2 \leq 1$, meaning they lie inside the unit circle.

4. Compute the Monte Carlo estimate of π :

$$\hat{\pi}_n = 4 \cdot \frac{K}{n}.$$

Task 1.

For a fixed $n = 1000$:

1. Generate two arrays `x` and `y` :

$$[x_1, \dots, x_n] \quad \text{and} \quad [y_1, \dots, y_n]$$

containing i.i.d. samples from the uniform distribution $U(-1, 1)$.

2. Create a boolean array `inside` of a size n that indicates whether each point (x_i, y_i) lies inside the unit circle. Similarly, create a boolean array `outside` for points lying outside the circle. Both should be computed without loops!
3. Compute the number of points inside the circle K and $\hat{\pi}_n$.
4. Visualize the results using:

```
import matplotlib.pyplot as plt

plt.title('\pi$ approximation')
plt.scatter(x[outside], y[outside], label='outside')
plt.scatter(x[inside], y[inside], label='inside')
plt.xlabel('$x_i$')
plt.ylabel('$y_i$')
plt.legend(loc='upper center', bbox_to_anchor=(0.5, -0.15), ncol=2)

print('K = ', K)
print('n = ', n)
print('pi approximation = ', pi_estimator)
```

```
In [6]: n = 1000
x = np.random.uniform(-1,1,n)
y = np.random.uniform(-1,1,n)
inside = np.where(x**2+y**2<1, True, False)
outside = np.where(x**2+y**2<1, False, True)
```

```
K = sum(inside)
pi_estimator = 4*K/n
# -----

import matplotlib.pyplot as plt

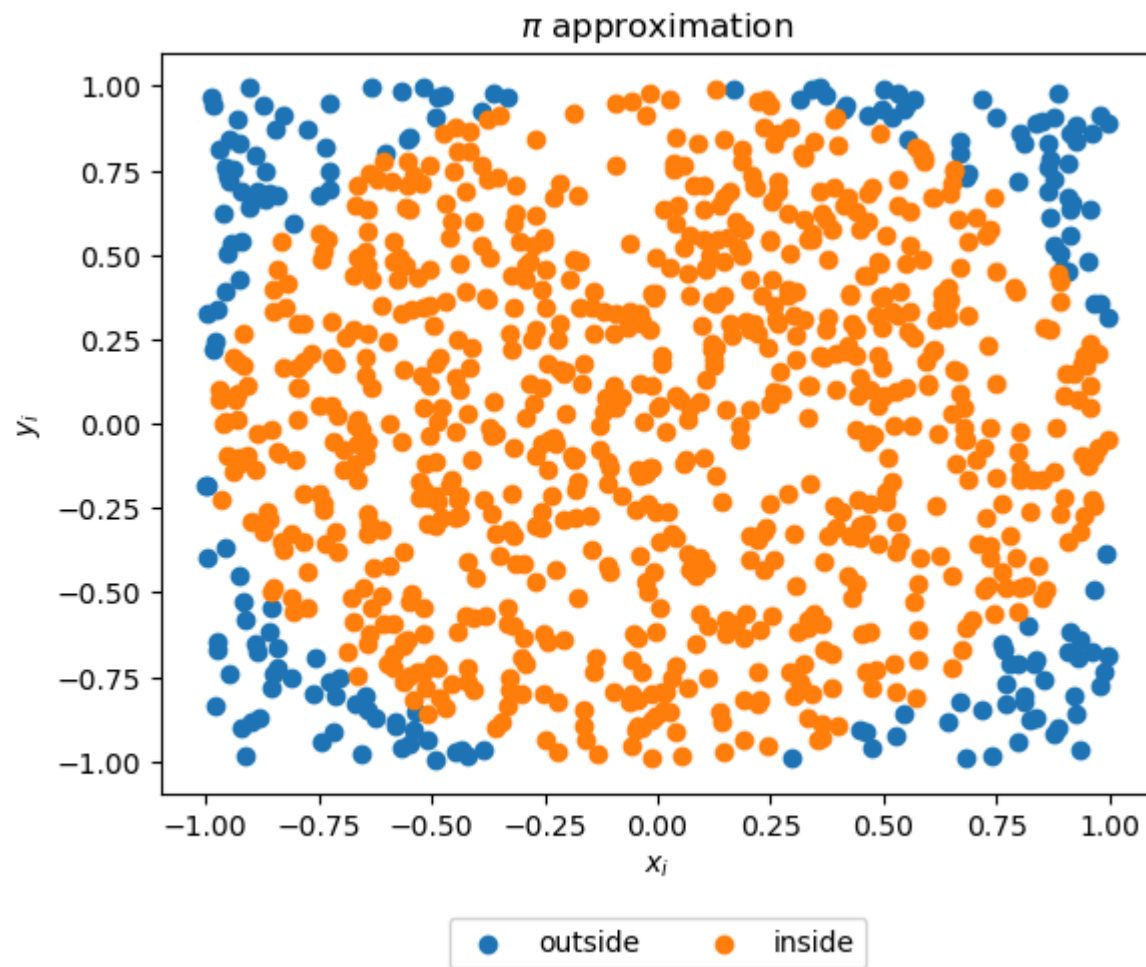
plt.title('$\pi$ approximation')
plt.scatter(x[outside], y[outside], label='outside')
plt.scatter(x[inside], y[inside], label='inside')
plt.xlabel('$x_i$')
plt.ylabel('$y_i$')
plt.legend(loc='upper center', bbox_to_anchor=(0.5, -0.15), ncol=2)

print('K = ', K)
print('n = ', n)
print('pi approximation = ', pi_estimator)
```

K = 804

n = 1000

pi approximation = 3.216



Task 2.

Since this mini-project will require using the π approximation algorithm multiple times, implement a loop-free, vectorized function `pi_approx` with parameter `n` representing the number of random points to be generated.

Evaluate it for `n=10000`.

```
In [15]: def pi_approx(n):  
         x = np.random.uniform(-1,1,n)  
         y = np.random.uniform(-1,1,n)  
         return 4*sum(np.where(x**2+y**2<1,1,0))/n
```

```
In [10]: pi_approx(10000)
```

Part 2 - Statistical properties

The value $\hat{\pi}_n$ is known as a Monte Carlo estimator for π . Mathematically, it has several important properties.

Note that

$$K \sim \text{Binomial}\left(\frac{\pi}{4}, n\right).$$

It can be shown that $\hat{\pi}_n$ is an unbiased estimator of π , meaning:

$$\mathbb{E}(\hat{\pi}_n) = 4np = \pi.$$

The variance of the estimator can also be computed:

$$\text{Var}(\hat{\pi}_n) = \frac{4^2 \cdot p(1-p)}{n} = \frac{4\pi \left(1 - \frac{\pi}{4}\right)}{n}.$$

Since the estimator is unbiased, the mean-squared error (MSE) equals the variance:

$$\text{MSE}(\hat{\pi}_n) = \mathbb{E}|\hat{\pi}_n - \pi|^2 = \text{Var}(\hat{\pi}_n) + \mathbb{E}^2[\hat{\pi}_n - \pi] = \frac{4\pi \left(1 - \frac{\pi}{4}\right)}{n}.$$

Taks 3.

Later on, we will validate the above results. Therefore, define a function `var_est(n)` that returns the theoretical variance of the Monte Carlo π approximation based on `n` random points.

In [185...

```
def var_est(n):
    return 4*np.pi*(1-np.pi/4)/n

print(f"Theoretical variance: {var_est(10000)}.")
```

Theoretical variance: 0.00026967662132698145.

Part 3 - Monte Carlo validation

We now validate the above theoretical results using Monte Carlo simulations.

Let M be the number of independent simulations. We perform the π approximation algorithm M times, obtaining:

$$\hat{\pi}_n^{(1)}, \hat{\pi}_n^{(2)}, \dots, \hat{\pi}_n^{(M)}.$$

Finally, we compute an estimate of the theoretical $\text{MSE}(\hat{\pi}_n)$:

$$\hat{\text{MSE}}(n, M) = \frac{1}{M} \sum_{m=1}^M |\hat{\pi}_n^{(m)} - \pi|^2.$$

Taks 3.

1. Define a function `MC_mse(n, M)` that:

- Performs `M` independent Monte Carlo simulations of the π -approximation algorithm, each using `n` random points.
- From $\hat{\pi}_n^{(1)}, \hat{\pi}_n^{(2)}, \dots, \hat{\pi}_n^{(M)}$ compute the empirical mean-squared error $\hat{\text{MSE}}(n, M)$ (without loops).
- Returns the empirical value $\hat{\text{MSE}}(n, M)$ and the theoretical value $\text{MSE}(\hat{\pi}_n)$.

2. Run `MC_mse(n=1000, M=1000)`.

```
In [20]: def MC_mse(n, M):  
  
    estimators = np.zeros(M)  
    for m in range(M):  
        estimators[m] = pi_approx(n)  
  
    mse_est = sum((estimators-np.pi)**2)/M  
    return mse_est, var_est(n)
```

```
In [90]: MC_mse(n=1000, M=1000)
```

```
Out[90]: (0.002677087044644287, 0.0026967662132698143)
```

Taks 4.

1. Run `MC_errors(n, M)` for $n \in \{2^5, 2^6, \dots, 2^{14}\}$ and $M = 500$.

2. Store the simulation-based results in `MSE_empirical` and the values from the theoretical formula in `MSE_theoretical`.
3. Compute the natural logarithms of arrays `N`, `MSE_empirical` and `MSE_theoretical`, naming them `logN`, `logMSE_empirical` and `logMSE_theoretical` (whout loops).
4. Plot the results for comparison using:

```
import matplotlib.pyplot as plt

plt.title('Error analysis')
plt.xlabel('$\log(n)$')
plt.ylabel('$\log(MSE)$')
plt.plot(logN, logMSE_empirical, 'go-', label='empirical')
plt.plot(logN, logMSE_theoretical, 'ro-', label='theoretical')
plt.legend()
plt.show()
```

```
In [118... import time      #i want to see how fast the code is, because i used "for" loop

start_time = time.time()
M=500
n_values=2**(np.array(range(10))+5)

results=np.array([MC_mse(n,M) for n in n_values])
MSE_empirical=results.T[0]
MSE_theoretical=results.T[1]

logN=np.log(n_values)
logMSE_empirical=np.log(MSE_empirical)
logMSE_theoretical=np.log(MSE_theoretical)

print(f"Time: {round((time.time() - start_time),4)} seconds.")    #on average, something like 3 seconds - not great, not terrible
```

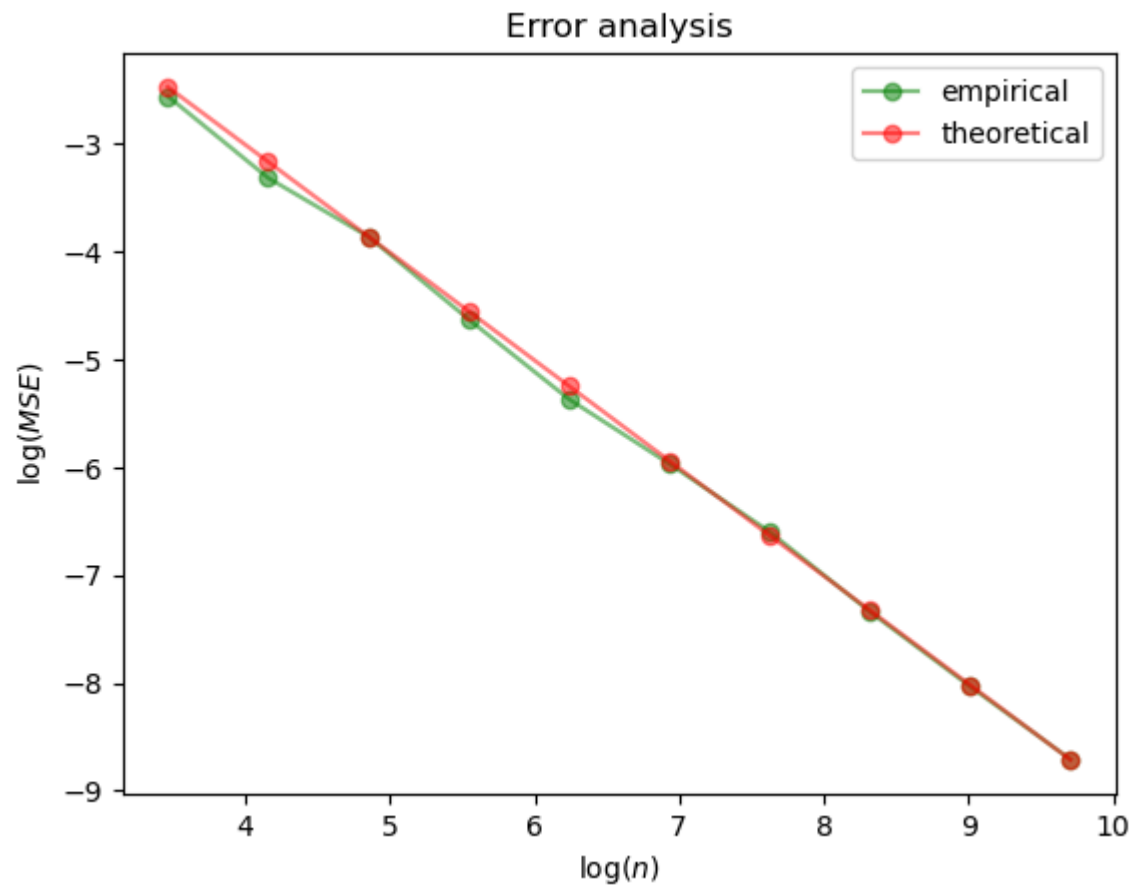
Time: 2.2596 seconds.

```
In [132... # -----

import matplotlib.pyplot as plt

plt.title('Error analysis')
plt.xlabel('$\log(n)$')
plt.ylabel('$\log(MSE)$')
plt.plot(logN, logMSE_empirical, 'go-', label='empirical', alpha=0.5)
plt.plot(logN, logMSE_theoretical, 'ro-', label='theoretical', alpha=0.5)
```

```
plt.legend()  
plt.show()
```



Part 4 - Rate of Convergence

We know that

$$MSE(\hat{\pi}_n) = Cn^{-1},$$

where $C > 0$ is a positive constant..

Let us now check whether the actual rate of convergence is indeed n^{-1} . For this purpose, we assume

$$MSE(\hat{\pi}_n) = Cn^{-a},$$

with some $a > 0$. Our goal is to obtain an empirical value of a and verify whether it is equal to 1.

Taking the logarithm of both sides gives

$$c_n = \log \text{MSE}(\hat{\pi}_n) = -a \log(n) - \log(C).$$

From this, we can derive formula for a

$$q_n = \frac{c_n - c_{n-1}}{\log(n) - \log(n-1)} = -a.$$

Thus, empirical values of q_n can be used to approximate a .

Task 5.

Without using loops, create an array `Q` whose elements are computed as

$$\hat{q}_n = \frac{\log \text{MSE}(\hat{\pi}_n) - \log \text{MSE}(\hat{\pi}_{n-1})}{\log(n) - \log(n-1)}$$

```
In [41]: Qs = (logMSE_empirical[1:]-logMSE_empirical[:-1])/(logN[1:]-logN[:-1])
print(Qs)    #they are indeed around the value -1

[-0.7911852  -1.11436612 -1.04090799 -0.79116463 -1.08859631 -1.01495812
 -0.87233021 -1.05809808 -1.21423247]
```

Part 5 - Confidence Intervals

Since

$$K \sim \text{Binomial}\left(\frac{\pi}{4}, n\right).$$

and

$$\hat{\pi}_n = \frac{4K}{n},$$

the estimator $\hat{\pi}_n$ is approximately normally distributed for large n by the Central Limit Theorem:

$$\hat{\pi}_n \overset{\text{approx}}{\sim} AN(\pi, \sigma_n^2),$$

where

$$\sigma_n^2 = \frac{4\pi \left(1 - \frac{\pi}{4}\right)}{n}.$$

In other words, we have

$$\frac{\hat{\pi}_n - \pi}{\sigma_n} \underset{\text{approx}}{\sim} N(0, 1).$$

Therefore,

$$P\left(-z_{1-\alpha/2} < \frac{\hat{\pi}_n - \pi}{\sigma_n} < z_{1-\alpha/2}\right) \approx 1 - \alpha,$$

where $z_{1-\alpha/2}$ is the $(1 - \alpha/2)^{\text{th}}$ percentile of the standard normal distribution.

Rearranging the inequality gives:

$$P\left(\hat{\pi}_n - z_{1-\alpha/2} \sigma_n < \pi < \hat{\pi}_n + z_{1-\alpha/2} \sigma_n\right) \approx 0.95.$$

This means that if we repeatedly generate independent Monte Carlo estimates $\hat{\pi}_n$, about $(1 - \alpha)100\%$ of the corresponding confidence intervals

$$\left[\underbrace{\hat{\pi}_n - z_{1-\alpha/2} \sigma_n}_{\hat{\pi}_{low}}, \underbrace{\hat{\pi}_n + z_{1-\alpha/2} \sigma_n}_{\hat{\pi}_{up}} \right]$$

will contain the true value of π .

Remark: The $(1 - \alpha/2)^{\text{th}}$ percentile of the standard normal distribution can be obtained using

```
import scipy.stats
z = scipy.stats.norm.ppf(1-\alpha/2)
```

Task 6.

Implement a function `ci_pi(n, alpha)` that returns a tuple $(\hat{\pi}_{low}, \hat{\pi}_{up})$ representing the lower and upper bounds of a $(1 - \alpha)100\%$ confidence interval for π .

Evaluate `ci_pi(n=1000, alpha=0.05)`.

In [183...

```
import scipy.stats

def ci_pi(n,alpha):
    pi_hat=pi_approx(n)
    z=scipy.stats.norm.ppf(1-alpha/2)
    sigma=np.sqrt(4*np.pi*(1-np.pi/4)/n)

    pi_low=pi_hat-z*sigma
    pi_up=pi_hat+z*sigma
    return((pi_low, pi_up))

print(f"{round((1-alpha)*100)}% confidence interval for pi: {ci_pi(n=1000, alpha=0.05)}")
```

90% confidence interval for pi: (3.066218290654385, 3.269781709345615).

Task 7.

Implement function `acp(n, alpha, M)` that empirically check the actual coverage probability of a $(1 - \alpha)100\%$ confidence interval for π . That is:

1. Run M independent simulations of the π approximation algorithm.
2. For each run, construct the 95% confidence interval using the formula above.
3. Count how many intervals contain π .
4. Compute

$$\text{ACP} = \frac{\text{Number of intervals containing } \pi}{M} * 100.$$

This empirical coverage APC should be close to $1 - \alpha$ if the normal approximation and variance formula are accurate.

In [104...

```
def acp(n, alpha, M):
    sim_intervals=np.array([ci_pi(n,alpha) for _ in range(M)])
    is_pi_there=np.where(sim_intervals[:,0]<=np.pi, np.where(np.pi<=sim_intervals[:,1],1,0),0) #sim_intervals[:,0]<=np.pi<=sim_interval[:,1]
    acp=sum(is_pi_there)/M*100
    return(acp)

print(f"We estimated pi right in the {round(acp(10000,0.05,500),2)}% of cases.")
```

We estimated pi right in the 94.2% of cases.

Task 8.

Compute the actual probability coverage (APC) with $M = 500$ for

$$\alpha \in \{0.01, 0.05, 0.1\}, \quad n \in \{2^5, 2^6, \dots, 2^{14}\}.$$

Store the results in a 3×10 array `results`, where each row corresponds to a fixed α and each column corresponds to a fixed n . Then, plot the results using:

```
plt.figure(figsize=(10,5))
plt.title('APC')

for i, alpha in enumerate(alphas):

    plt.plot(results[i, :],
             label = 'alpha = ' + str(alpha),
             color = 'C' + str(i),
             linestyle = '-',
             marker='o')

    plt.hlines((1-alpha)*100,
              xmin = 0,
              xmax = len(N),
              color = 'C' + str(i),
              linestyle = '--')

plt.legend()
plt.xlabel('$n$')
plt.xticks(np.arange(len(N)), N, rotation = 90)
plt.show()
```

In [120...

```
start_time = time.time()
alphas=np.array([0.01, 0.05, 0.1])
N=2**(np.array(range(10))+5)

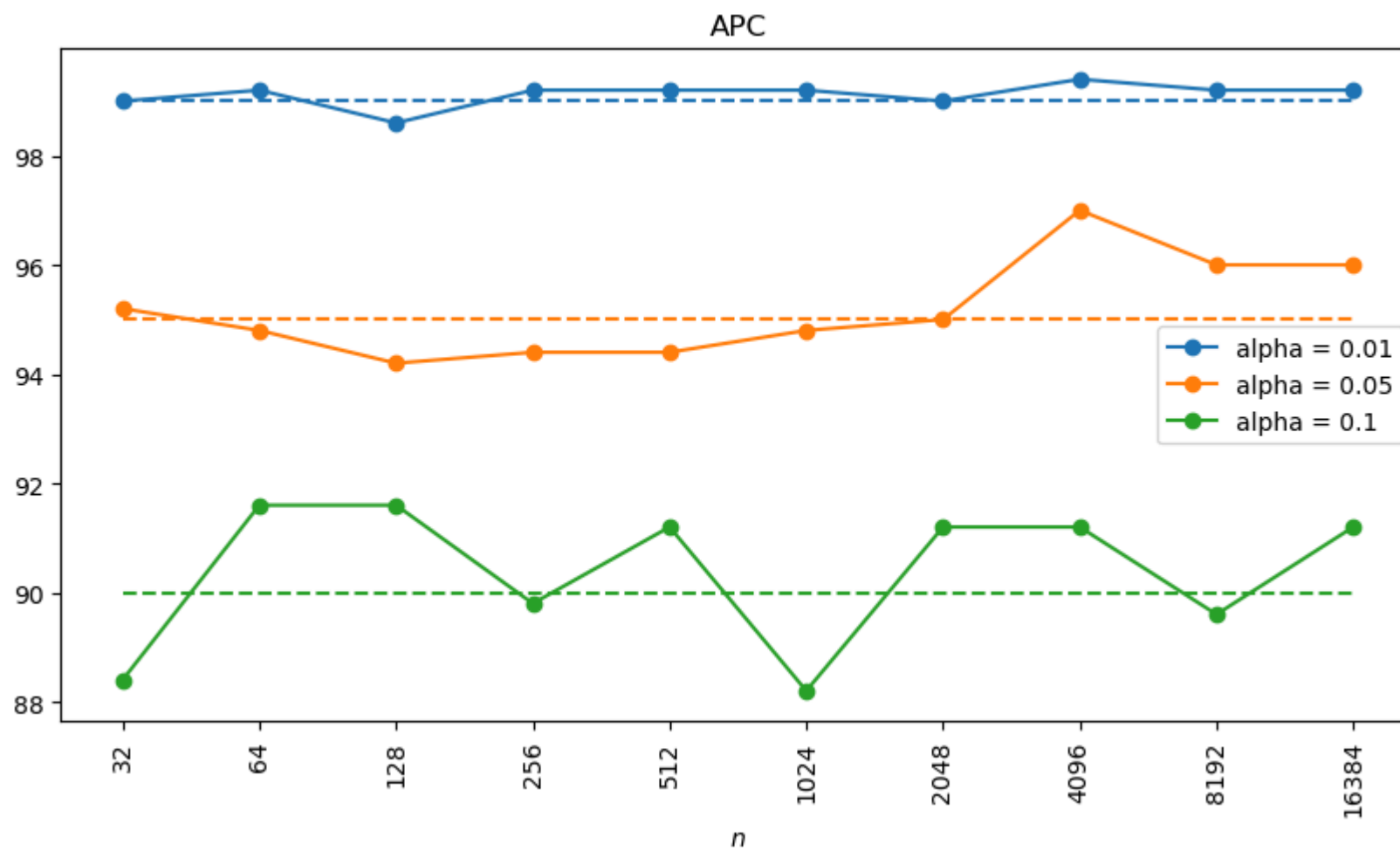
results=np.zeros((len(alphas), len(N)))
for ind_n, n in enumerate(N): #is it okay to leave for loop here? it only goes through 30 elements...
    for ind_a, a in enumerate(alphas):
        results[ind_a][ind_n]=acp(n,a,500)

print(results)
print(f"Time: {(time.time() - start_time)} s.")
```

```
[[99.  99.2 98.6 99.2 99.2 99.2 99.  99.4 99.2 99.2]
 [95.2 94.8 94.2 94.4 94.4 94.8 95.  97.  96.  96. ]
 [88.4 91.6 91.6 89.8 91.2 88.2 91.2 91.2 89.6 91.2]]
Time: 12.631178617477417 s.
```

In [121...

```
# -----  
  
plt.figure(figsize=(10,5))  
plt.title('APC')  
  
for i, alpha in enumerate(alphas):  
  
    plt.plot(results[i, :],  
             label='alpha = ' + str(alpha),  
             color='C' + str(i),  
             linestyle='--',  
             marker='o')  
  
    plt.hlines((1-alpha)*100,  
              xmin = 0,  
              xmax = len(N)-1,  
              color='C' + str(i),  
              linestyle='--')  
  
plt.legend()  
plt.xlabel('$n$')  
plt.xticks(np.arange(len(N)), N, rotation = 90)  
plt.show()
```



Multi-Dimensional π approximation

We generalize the π -approximation method to estimate the volume of a unit d -dimensional sphere (also called a hypersphere).

The volume of a unit sphere in d dimensions is:

$$V_d = \frac{\pi^{d/2}}{\Gamma(d/2 + 1)},$$

where $\Gamma(\cdot)$ is the gamma function.

Remark

To compute $\Gamma(d/2 + 1)$ you can use

```
from scipy.special import gamma
gamma(d/2+1)
```

Task 9.

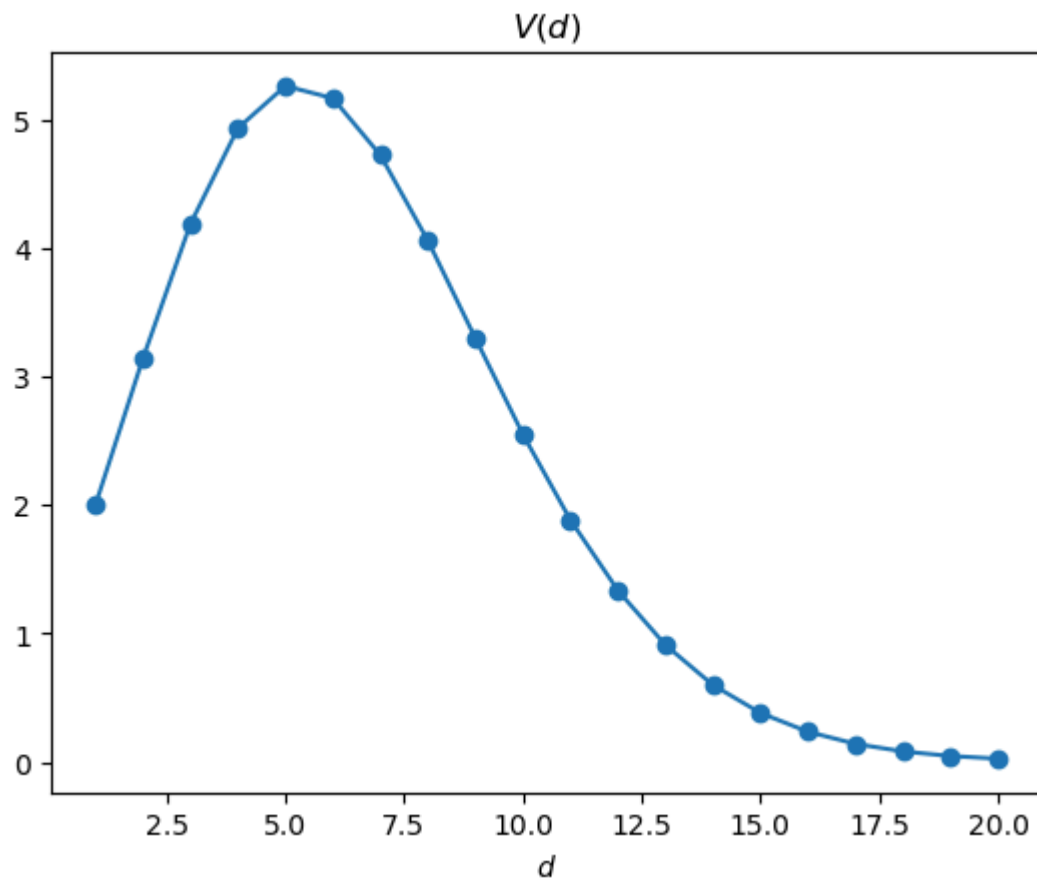
For a given array `D = np.arange(1, 20+1)` create, in one line and without loops, an array `volumes` containing the exact volumes of a unit sphere in dimension d in `D`. Then, plot the results using:

```
plt.title('$V(d)$')
plt.plot(D, volumes, 'o-')
plt.xlabel('$d$')
plt.show()
```

In [126...

```
from scipy.special import gamma
D=np.arange(1,21)
volumes=np.pi**(D/2)/gamma(D/2+1)

plt.title('$V(d)$')
plt.plot(D, volumes, 'o-')
plt.xlabel('$d$')
plt.show()
```



Assume that the sphere is inscribed in a hypercube $[-1, 1]^d$, that is

$$\{(x_1, \dots, x_d) \in [-1, 1]^d : x_1^2 + \dots + x_d^2 \leq 1\}.$$

If we generate points uniformly in the hypercube and count the fraction that lies inside the sphere multiplying by the hypercube volume 2^d yields the estimate of

$$V_d = \frac{\pi^{d/2}}{\Gamma(d/2 + 1)}.$$

Task 10.

Implement a vectorized function `hypersphere_volume(d, n)` that:

- Generates `n` random points in `d` dimensions from $U(-1, 1)$ (using one line).

- Counts how many (say K) lie inside the unit sphere (using one line).
- Returns the Monte Carlo estimate of V_d :

$$\hat{V}_d = \frac{2^d K}{n}$$

Evaluate `hypersphere_volume(2, 10000)`, `hypersphere_volume(5, 10000)`, `hypersphere_volume(9, 10000)`. Compare them with the theoretical values.

```
In [152... def hypersphere_volume(d,n):
    points=np.random.random((n, d))*2-1 #shift from [0,1) to [-1,1)
    K=sum(np.where(sum(points.T**2)<=1,1,0))
    return(2**d*K/n)
```

Out[152... 5.4656

```
In [179... print(f"V_hat_2 = {hypersphere_volume(2, 10000)}, theoretical volume: V_2 = {volumes[1]}, relative error: {round(abs(hypersphere_volume(2,
print(f"V_hat_5 = {hypersphere_volume(5, 10000)}, theoretical volume: V_5 = {volumes[4]}, relative error: {round(abs(hypersphere_volume(5,
print(f"V_hat_9 = {hypersphere_volume(9, 10000)}, theoretical volume: V_9 = {volumes[8]}, relative error: {round(abs(hypersphere_volume(9,
```

V_hat_2 = 3.1284, theoretical volume: V_2 = 3.141592653589793, relative error: 0.18%.
V_hat_5 = 5.296, theoretical volume: V_5 = 5.263789013914324, relative error: 1.64%.
V_hat_9 = 3.4816, theoretical volume: V_9 = 3.298508902738706, relative error: 6.87%.